Real-Time Lane Detection and Autonomous Steering

A comparison of classical pipelines and a polynomial-fit improvement

David Tovmasyan

Course project, May 2026

Where we are going

Motivation

The simulator

Three lane detectors

Two controllers

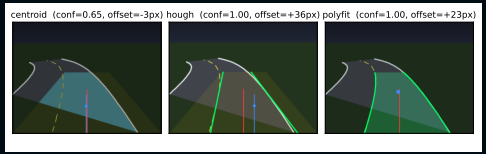
Experiments

Live demo

Discussion and conclusion

Why lane keeping (still) matters

- Most-deployed AD function: every modern car ships lane-keep assist or adaptive cruise.
- Two coupled problems
  - **Perception:** where is the lane?
  - **Control:** how do we stay in it?
- Learned segmenters dominate benchmarks, but a classical pipeline is:
  - interpretable (debuggable),
  - CPU-friendly (> 500 FPS),
  - zero training data required.



Same frame, three detectors.

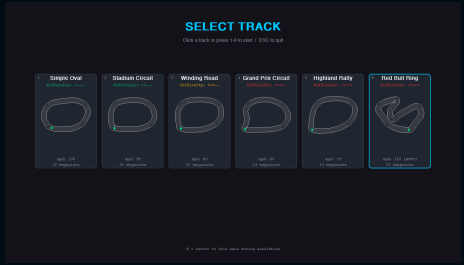
The concrete question

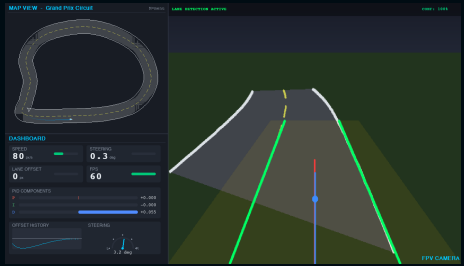
How much performance is left on the table
by the textbook Canny+Hough detector,
and can a **polynomial-fit upgrade**
close the gap on curved tracks
while staying **real-time**?

- Implement three classical methods of increasing sophistication.
- Plug each into a PID controller.
- Add a Stanley controller driven from *ground truth* as a perception-free oracle (upper bound on what the controller can do given perfect perception).
- Evaluate on five tracks + a noise sweep.

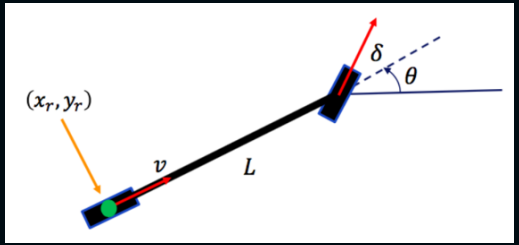
What we built

- 2D forward-perspective driving simulator (Python + OpenCV + pygame).
- 5 tracks (difficulty 1–5), closed-loop Catmull–Rom spline.
- Camera: pinhole, 70° FoV, 18° pitch, 640×480 FPV.
- Vehicle: bicycle kinematics, $\dot{\theta} = (v/L)\tan\delta$.
- Three live panels: map, FPV overlay, dashboard.





Vehicle model: bicycle kinematics



Rear-axle reference point (x_r, y_r), forward speed v, wheelbase L, heading θ, front-wheel steering angle δ.

Continuous-time kinematics integrated each frame:

$$\dot{x}_r = v\cos\theta$$

$$\dot{y}_r = v\sin\theta$$

$$\dot{\theta} = \frac{v}{L}\tan\delta$$

- Front-wheel steering, rear-axle reference – the standard simplification for lane-keeping work.
- δ clamped to $\pm 40°$; Δt capped at 50 ms to keep the Euler integration stable.
- No slip, no tire model – realistic enough for highway- speed lane keeping, deliberately simple so the comparison is about perception, not dynamics.

Ground truth = analytical

Because we own the simulator, we can compute the things a benchmark is normally missing:

- **True lateral offset** from the centreline spline.
- **True heading error** vs. the track tangent.
- **True curvature** ahead (adaptive speed control, turn-sharpness grouping).
- **Ground-truth lane polygon** in the camera frame (re-project road from spline) $\Rightarrow$ **lane-IoU** against any predicted mask.

$\Rightarrow$ Every metric is anchored to a number the detector *cannot see*.

Method 1: centroid (our baseline)

Pipeline

1. Threshold the grey road colour in HSV.
2. Mask to a trapezoidal ROI.
3. Centroid of the blob (image moments).
4. EMA smoothing across frames.

Role

- Simplest detector we could write $\Rightarrow$ adopted as the **reference baseline**.
- Tells us how much the lane-line geometry of Methods 2–3 actually buys.

Spoiler

Does not complete a lap on any track.

The blob centroid drifts off-centre whenever the road is asymmetric in the field of view.

Method 2: Hough lines (the textbook pipeline)

Pipeline

1. Blur, then Canny edges.
2. Trapezoidal ROI – keep only the road.
3. Probabilistic Hough $\Rightarrow$ line segments.
4. Split by slope sign: left vs. right lane.
5. Length-weighted average $\Rightarrow$ one line per side.
6. Lane centre = midpoint at the bottom.

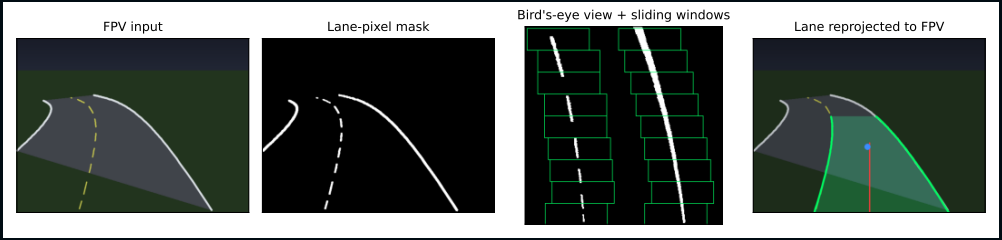
Strength

- Fast, no training, fully interpretable.
- Rock-solid on straight road.

Weakness

- **Models every lane as a straight line.**
- In a curve, the line cuts the corner – the error grows with the bend.

Method 3: bird's-eye + sliding-window polyfit



Four stages, each fixing a specific weakness of Method 2:

- **Lane pixels** – colour + edges, not just Canny.
- **Bird's-eye warp** – the model sees true road shape.
- **Sliding windows** – robust on dashed lines.
- **Quadratic fit** – a curve, not a straight line.

Method 3, step 1 – undo the perspective

Problem. In the camera image, the same physical dash covers ~ 30 px near the car but only ~ 2 px far ahead. A pixel-counting curve fit therefore listens to the near road and **ignores the far field** – exactly the part that announces upcoming curves.

Fix. A fixed homography warps the ROI trapezoid into a square top-down view. In that view:

- Every dash is the same size, near or far.
- Curved lanes become **gentle bends**, not vanishing-point lines.
- A 2nd-order polynomial $x = ay^2 + by + c$ fits a typical road curve almost exactly.

Why this is the key step

Without the warp, even a quadratic fit barely beats Hough, because the near road still dominates.

With the warp, near and far pixels get **equal say** in the fit.

Method 3, step 2 – find each lane

Inside the bird's-eye view, which pixels belong to the left vs. right lane?

Sliding-window search

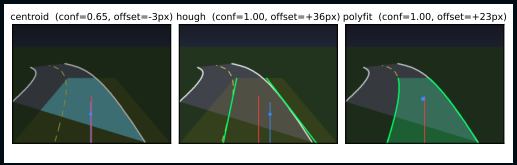
1. **Seed**: column histogram of the bottom half $\Rightarrow$ two strongest peaks = lane bases.
2. **Walk upward**: stack of 9 rectangular windows above each base; collect bright pixels inside.
3. **Recentre**: if enough pixels are found, slide the next window over to follow them. Robust to dashed lines and short gaps.

Quadratic fit + reprojection

- Least-squares fit $x = ay^2 + by + c$ on the collected pixels, one polynomial per lane.
- EMA on the coefficients $\Rightarrow$ stable across frames.
- Warp the curve back to the camera image for the overlay and for the lane-IoU metric.

Method 3, step 3 – the look-ahead offset

- The polynomial gives us the lane *shape*, not just one point on it.
- Evaluate the lane centre at **45 % up the BEV** – a 1-frame predictor, essentially free.
- Feed *that* to the PID instead of the bottom-of- image offset.
- Turn it off and the polyfit's RMS gain shrinks from $\sim 40\,\%$ to $\sim 10\,\%$.



The blue dot in the right panel is the reprojected look-ahead point.

PID – the baseline controller

$$\delta_t = K_p\, e_t + K_i \sum_{\tau \le t} e_\tau\, \Delta t + K_d\, \dot{e}_t$$

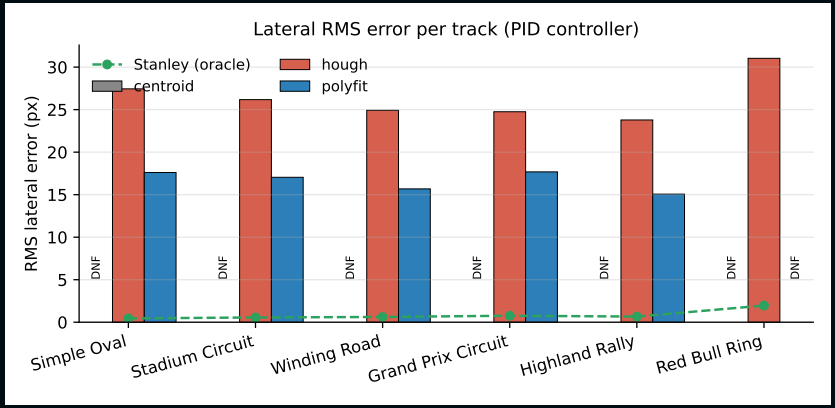
- Input: normalised lateral offset $e \in [-1, +1]$ from the detector.
- Gains tuned by hand on the oval: $K_p = 1.2$, $K_i = 0.008$, $K_d = 0.5$.
- Integral anti-windup at ± 50.
- Reactive only – has no model of the path.

Stanley – the perception-free oracle

$$\delta = \psi + \arctan\left(\frac{k\, e_\perp}{k_s + v}\right)$$

- Heading error ψ + cross-track error $e_\perp$ at the front axle, both computed from the *true* spline.
- Gains $k = 0.6$, $k_s = 20$.
- **It does not look at the camera.**
- Result on every track: < 1 px RMS lateral error.
- Interpretation: the gap between PID-on-detection and Stanley-on-truth is *purely* the perception error.

Headline result: RMS lateral error per track



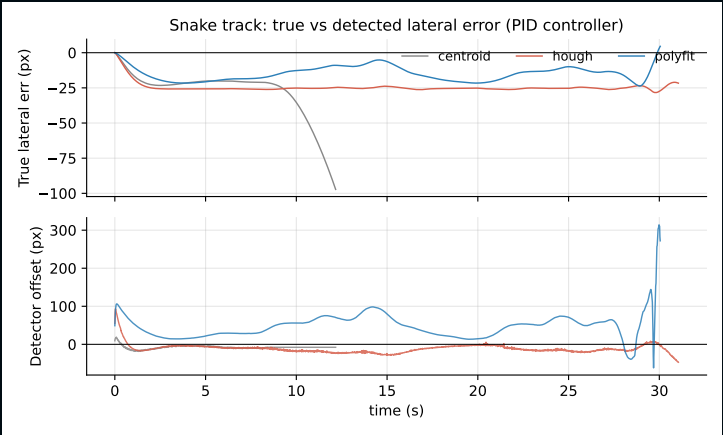
Centroid **DNF** on every track. On the 5 base tracks: Hough 25.4 px mean RMS, polyfit **16.6** px (**-34 %**). Red Bull Ring (hairpins, adaptive speed): **polyfit DNFs** at a hairpin, Hough survives at 31.0 px. Stanley+truth: < 2 px throughout.

The full table

Track	Detector	Lap	RMS px	Mean px	Max px	IoU	ms/det
Simple Oval	centroid	DNF	99.5	80.0	194.3	0.01	1.15
Simple Oval	hough	✓	27.4	27.2	28.7	0.17	0.75
Simple Oval	polyfit	✓	17.6	16.9	27.2	0.25	1.92
Stadium Circuit	centroid	DNF	39.5	32.8	113.4	0.39	1.79
Stadium Circuit	hough	✓	26.2	26.0	30.1	0.24	0.77
Stadium Circuit	polyfit	✓	17.0	16.3	23.6	0.31	1.94
Winding Road	centroid	DNF	33.9	28.2	97.8	0.41	1.84
Winding Road	hough	✓	24.9	24.7	28.4	0.23	0.76
Winding Road	polyfit	✓	15.7	14.8	23.7	0.35	1.96
Grand Prix Circuit	centroid	DNF	35.0	28.4	108.4	0.43	1.91
Grand Prix Circuit	hough	✓	24.8	24.6	27.8	0.26	0.77
Grand Prix Circuit	polyfit	✓	17.7	16.1	34.5	0.36	1.95
Highland Rally	centroid	DNF	65.6	52.9	127.6	0.00	1.22
Highland Rally	hough	✓	23.8	23.6	26.1	0.31	0.78
Highland Rally	polyfit	✓	15.1	14.2	21.9	0.39	1.95
Red Bull Ring	centroid	DNF	117.7	95.0	228.4	0.01	1.12
Red Bull Ring	hough	✓	31.0	30.5	51.9	0.21	0.73
Red Bull Ring	polyfit	DNF	28.5	22.8	82.0	0.24	1.90
Simple Oval	Stanley (oracle)	✓	0.4	0.4	1.2	0.52	2.04

- Where polyfit completes, it beats Hough on RMS and IoU ($-34\,\%$ RMS, $+38\,\%$ IoU averaged).
- **Red Bull Ring caveat**: the quadratic fit briefly blows up at a hairpin and the car leaves the road. Hough's straight line degrades gracefully and finishes.

Winding Road: where polyfit pays off

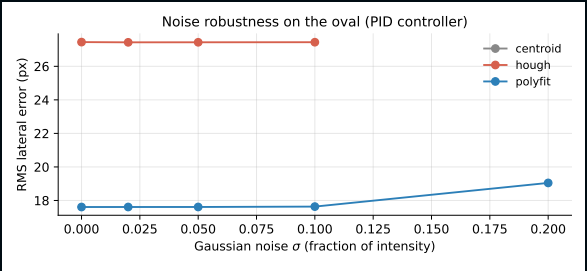


Top: true lateral error – centroid bails by $t \approx 12\,\text{s}$; Hough sits at $-25\,\text{px}$ through the S-curves; polyfit oscillates around 0 (follows bends, doesn't cut them).
Bottom: the detector offset fed to the PID – both polyfit and Hough stay well-behaved on this track.

Red Bull Ring: where polyfit falls over

Top: centroid bails at $t \approx 3$ s. Hough holds ~ 30 px to the flag. Polyfit trails Hough until $t \approx 17$ s.
Bottom: a hairpin makes the quadratic fit explode to ~ 8000 px for two frames – PID over-corrects, car off-track. **Fix:** clamp detector offset, or reject fit outliers.

Robustness to camera noise



- Same oval, PID, +Gaussian noise $\sigma \in [0, 0.20]$ per frame.
- Centroid never completes.
- Hough survives $\sigma \leq 0.10$, fails at 0.20 (Canny drowned out).
- **Polyfit completes at every σ** – its mask uses a *normalised* Sobel-x + colour.

Compute cost

Detector	ms / frame	equiv. FPS
centroid	1.7	$\sim$600
hough	0.7	$\sim$1450
polyfit	1.9	$\sim$520

- All three are comfortably real-time on a single CPU thread.
- Polyfit overhead is dominated by the perspective warp and `np.polyfit`, not by the sliding-window scan.
- Plenty of budget left for downstream tasks (other vehicles, obstacles, signs).

Live demo

```
python main.py --detector polyfit --controller pid
```

- Press **D** to cycle detectors live.
- Press **C** to toggle PID $\leftrightarrow$ Stanley.
- Press **M** to jump back to the track selector.
- Watch the dashboard: lane offset, FPS, PID components.

Suggested demo route: *Stadium* (Hough wobbles) $\rightarrow$ *Winding Road* (Hough struggles) $\rightarrow$ switch to polyfit $\rightarrow$ switch to Stanley.

What worked, what failed

Worked

- Look-ahead offset = single biggest gain.
- HSV + Sobel mask cleaner than Canny on dashed lines.
- Quadratic fits + EMA give stable curves.

Failed / limitations

- Centroid is structurally biased – no tuning saves it.
- Hough lingers on its last good line through tight bends.
- Polyfit's quadratic fit can **blow up at tight hairpins** (Red Bull Ring) – needs outlier rejection.
- IPM warp assumes flat ground; real cameras need re-tuning.
- Simulator is too clean – numbers are an upper bound.

Take-aways

- A modest classical-CV upgrade (BEV warp + sliding-window quadratic fit) cuts RMS lateral error by **34–40 %** vs. the textbook Hough pipeline.
- The change is fully interpretable: we can point at each stage in Fig. 3 and say why it helps.
- Look-ahead offsets are essentially free on top of a polynomial fit and turn a reactive PID into a quasi-MPC.
- Closing the remaining gap to Stanley-on-truth would need sub-pixel-accurate perception $\Rightarrow$ a learned segmenter is the next step.

Thank you.

Code, simulator, report and these slides are all in this repo.
Questions?

Backup: detector confidence per track

- Centroid mean confidence: **0.27** (HSV blob coverage, near zero whenever the road leaves the trapezoid).
- Hough mean confidence: **0.99** (both lines almost always found – but "found" $\neq$ "correct").
- Polyfit mean confidence: **1.00** (both polynomial fits valid in every frame after warm-up).
- Take-away: high detector confidence does not imply low error. Lane-IoU is the better proxy.

Backup: why the look-ahead works

Recall the bicycle heading update:

$$\dot{\theta} = \frac{v}{L} \tan \delta$$

- A pure-P controller on e_{bottom} reacts to *current* position only.
- Adding the look-ahead error e_{LA} effectively augments the controller with a free derivative term that knows where the road is going.
- Equivalent to a one-step preview MPC with horizon $\sim 0.5\,\text{s}$ at typical speeds – without solving an optimisation.

Backup: reproducibility

```
git clone <repo>
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt matplotlib pdf2image
python experiments.py   # CSVs + summary figures
python make_qualitative_figures.py   # overlays
cd presentation && tectonic report.tex
cd presentation && tectonic slides.tex
```

All experiments are deterministic except the noise sweep, seeded by NumPy's default RNG.